

Tugas P9 Pemrograman Berorientasi Objek

Nama : Torikh Abdullah Naser

NIM : 2410512031

Kelas : A-S1 Sistem Informasi

Analisis Polimorfisme pada Class Enemy

Berdasarkan implementasi kode, berikut adalah hasil analisisnya:

1. Mengapa hasil output bisa berbeda walaupun method yang dipanggil sama?

Hasil output berbeda karena adanya konsep **Dynamic Method Binding** (atau Late Binding).

Meskipun dalam loop kita menggunakan referensi tipe induk (Enemy e), Java Virtual Machine (JVM) akan memeriksa **tipe objek sebenarnya** (actual object type) pada saat runtime (saat program berjalan).

- Jika objek tersebut adalah instance dari Zombie, maka method attack() milik Zombie yang dijalankan.
- Jika objek tersebut adalah instance dari Pocong, maka method attack() milik Pocong yang dijalankan.

Hal ini dimungkinkan karena setiap subclass telah melakukan **Overriding** terhadap method attack() dari class induknya.

2. Ini termasuk jenis polimorfisme apa?

Ini termasuk dalam jenis **Runtime Polymorphism** (atau Dynamic Polymorphism).

Karakteristik utamanya adalah:

- Terjadi pada saat program sedang berjalan (Runtime).
- Dicapai melalui mekanisme **Method Overriding**.
- Memungkinkan sebuah referensi tipe induk untuk merujuk ke objek tipe anak dan menjalankan perilaku yang spesifik sesuai dengan objek tersebut.

IMPLENTASI KODE

1. File Enemy.java

```
p9 >  Enemy.java > ...  
1 package p9;  
2  
3 public class Enemy {  
4     public void attack() {  
5         System.out.println("Enemy is attacking!");  
6     }  
7 }  
8 | Ctrl+L for Agent
```

2. File Zombie.java

```
p9 >  Zombie.java > ...  
1 package p9;  
2  
3 public class Zombie extends Enemy {  
4     @Override  
5     public void attack() {  
6         System.out.println("Zombie attacks: Chomp chomp! (Eating brains)");  
7     }  
8 }  
9 |
```

3. File Pocong.java

```
p9 >  Pocong.java > ...  
1 package p9;  
2  
3 public class Pocong extends Enemy {  
4     @Override  
5     public void attack() {  
6         System.out.println("Pocong attacks: Jump and headbutt!");  
7     }  
8 }  
9 |
```

4. File Burung.java

```
p9 >  Burung.java > ...  
1 package p9;  
2  
3 public class Burung extends Enemy {  
4     @Override  
5     public void attack() {  
6         System.out.println("Burung attacks: Pecking from above!");  
7     }  
8 }  
9 | Ctrl+L for Agent
```

5. File Main.Java

```
p9 > Main.java > ...
1  package p9;
2
3  import java.util.ArrayList;
4  import java.util.List;
5
6  public class Main {
7      public static void main(String[] args) {
8          List<Enemy> enemies = new ArrayList<>();
9
10         enemies.add(new Zombie());
11         enemies.add(new Pocong());
12         enemies.add(new Burung());
13         enemies.add(new Enemy());
14
15         System.out.println("--- Enemy Attack Simulations ---");
16
17         for (Enemy e : enemies) {
18             e.attack();
19         }
20     }
21 }
```

6. Output kode

```
PS D:\kuliah\smt 4\prak pbo> d:; cd 'd:\kuliah\smt 4\prak pbo'; java -cp 'p9.Main'
--- Enemy Attack Simulations ---
Zombie attacks: Chomp chomp! (Eating brains)
Pocong attacks: Jump and headbutt!
Burung attacks: Pecking from above!
Enemy is attacking!
PS D:\kuliah\smt 4\prak pbo>
```

Source Code: <https://github.com/Torikh42/pbo>